SMART WIFI SURVEILLANCE SYSTEM

Designed and Developed by

1. D. Nandhitha Reddy 2211CS010149
2. D. Varsha Reddy 2211CS010135
3. Brendan Phillips 2211CS010092
4. CH. Shiva 2211CS010119

Guided by

Mr. K. Sreekanth
Assistant Professor

Department of Computer Science & Engineering

MALLA REDDY UNIVERSITY, HYDERABAD

2024-2025

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

CERTIFICATE

This is to certify that this is the Application development lab record entitled "**SMART WIFI SURVEILLANCE SYSTEM**", submitted by **D. NANDHITHA REDDY -2211CS010149, D. VARSHA REDDY -2211CS010135, BRENDAN PHILLIPS -2211CS010092, CH. SHIVA -2211CS010119** B. Tech **III** year I semester, Department of CSE during the year 2024-25. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide
Mr. K. Sreekanth
(Assistant Professor)

Dean-CSE
Mr. Shaik Meeravalli

External Examiner

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

DECLARATION

We declare that this project report titled **SMART WIFI SURVEILLANCE SYSTEM** submitted in partial fulfillment of the degree of B. Tech in CSE is a record of original work carried out by me under the supervision of **MR. K. SREEKANTH (ASSISTANT PROFESSOR)** and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Signature

1. D. Nandhitha Reddy 2211CS010149
2. D. Varsha Reddy 2211CS010135
3. Brendan Phillips 2211CS010092
4. CH. Shiva 2211CS010119

ACKNOWLEDGEMENTS

We have been truly blessed to have a wonderful internal guide **Mr. K. Sreekanth (Assistant Professor)**, Department of CSE, **Malla Reddy University** for guiding us to explore the ramification of our work and we express our sincere gratitude towards her for leading me through the completion of Project.

We would like to say our sincere thanks to **Mr. G. Mohan Ram**, Department of CSE, App Development Coordinator, for providing seamless support and right suggestions are given in the development of the App.

We would like to say our sincere thanks to **Mr. Shaik Meeravalli** Dean, Department of CSE I. B.Tech, **Malla Reddy University** for providing seamless support and right suggestions are given in the development of the App.

We wish to express our sincere thanks to **Vice Chancellor** sir and The Management of **Malla Reddy University** for providing excellent infrastructure and their visionary thoughts to prepare ourselves industry ready by focusing on new technologies.

Finally, we would like to thank our family members and friends for their moral support and encouragement to achieve goals.

ABSTRACT

The Smart WiFi Surveillance System is an advanced project designed to revolutionize vehicle interaction through enhanced connectivity, real-time monitoring, and automated control. At its core is the ESP32 WiFi module, which provides seamless communication between the vehicle and external networks, allowing users to remotely access and control their vehicle through a web interface. This connectivity makes vehicle management more convenient and user-friendly.

The integration of the ESP32 Camera Module offers continuous live video surveillance, enabling real-time monitoring of the vehicle's surroundings. This feature significantly enhances security by allowing owners to keep an eye on their vehicle at any moment, ensuring swift detection of suspicious activities or potential threats.

A key aspect of the system is the use of an L298 motor driver connected to four motors, enabling precise and flexible remote navigation. This design allows users to control the vehicle's movement with ease, providing added convenience for a range of uses. The combination of video surveillance and motorized control ensures a well-rounded tool for vehicle interaction.

Overall, the Smart WiFi Surveillance System offers a comprehensive solution for vehicle security, remote monitoring, and control. With its advanced features and user-centric design, this system is ideal for vehicle owners seeking enhanced safety and control through a smart, connected experience.

TABLE OF CONTENT

DESCRIPTION	PAGE NUMBER
CERTIFICATE	I
DECLARATION	II
ACKNOWLEDGEMENTS	III
ABSTRACT	IV
Chapter 1 Introduction	
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objective of Project	3
1.4 Goal of Project	4
Chapter 2 Problem Identification	
2.1 Existing System	5
2.2 Proposed System	6
Chapter 3 Requirements	
3.1 Software Requirements	8
3.2 Hardware Requirements	8
Chapter 4 Design and Implementation	
4.1 Design	11
4.2 Implementation	15
Chapter 5 Code	
5.1 Source Code	18
5.2 Screenshot of Application	32

Chapter 6 Results & Conclusion

6.1 Results **34**

6.2 Conclusion **35**

REFERENCES **36**

CHAPTER – 1

INTRODUCTION

1.1 Introduction

As technology advances, the automotive sector continues to adopt innovations that enhance connectivity and security. The Smart WiFi Surveillance System marks a significant leap in this space, redefining vehicle interaction through advanced automation. This project is engineered to transform how vehicles communicate with their surroundings and owners, utilizing state-of-the-art technology to create a seamless user experience.

At the core of this system lies the ESP32 WiFi module, which ensures uninterrupted communication between the vehicle and external networks. This connectivity facilitates real-time monitoring and remote control via a web server, allowing users to easily access and manage their vehicle from any location. This feature adds a layer of convenience that aligns with modern expectations for smart vehicle management.

The integration of the ESP32 Camera Module further enhances the system by enabling continuous video surveillance. This allows users to monitor their vehicle's surroundings at all times, bolstering security and providing immediate awareness of any potential issues. The ability to view live video feeds empowers vehicle owners to maintain better oversight and protection.

Additionally, the L298 Motor Driver connected to four motors supports precise and dynamic movement, giving users flexible remote navigation control. This functionality enriches the system by allowing the vehicle to respond seamlessly to commands, making it suitable for various use cases. The Smart WiFi Surveillance System sets a new standard for automotive technology by combining intelligent connectivity with advanced security and control, ensuring a safer and more connected driving experience.

1.2 Problem Statement

As automotive technology evolves, there is an increasing demand for advanced solutions that deliver enhanced vehicle security, real-time monitoring, and remote control capabilities. Traditional security systems, such as alarms and immobilizers, provide only basic protection and fall short of offering live surveillance or the ability to control vehicle movement. While vehicle tracking systems can provide location data, they lack comprehensive real-time monitoring and remote interaction features, leaving gaps in security, especially in the face of increasing vehicle theft and unauthorized access.

With the rise of connected and autonomous vehicles, seamless integration with external networks that enable remote monitoring and control has become essential. However, many existing systems do not effectively combine features like live video surveillance and dynamic vehicle movement control in a unified platform. This absence of integration leads to inefficiencies and potential security vulnerabilities that can compromise vehicle safety and user convenience.

The challenge is to create a robust solution that provides real-time vehicle monitoring and control while maintaining strong security measures against unauthorized access. Vehicle owners require an intuitive system that enables them to monitor and control their vehicles from a distance, providing peace of mind and greater situational awareness without complexity.

The Smart WiFi Surveillance System rises to meet these challenges by integrating advanced components such as the ESP32 WiFi module, the ESP32 Camera Module, and the L298 motor driver into a cohesive, intelligent platform. This system allows for continuous live video surveillance, real-time vehicle control, and seamless remote interaction. By combining these technologies, it offers an all-in-one solution for enhanced vehicle security and user experience, supporting a smarter, more connected approach to vehicle management.

1.3 Objectives

The objective of the Smart WiFi Surveillance System is to develop an advanced, connected automotive solution that enhances vehicle security, control, and monitoring capabilities through seamless integration of IoT technologies. Specifically, the system aims to:

- **Facilitate Real-Time Monitoring and Control:** Utilize the ESP32 WiFi module to enable seamless communication with a web server, allowing users to monitor live video feeds and control vehicle functions remotely.
- **Promote Smart Vehicle Connectivity:** Establish a foundation for IoT-enabled connectivity, creating a more intelligent and responsive vehicle system that enhances user experience and interaction.
- **Enhance Vehicle Security:** Develop a continuous surveillance system that allows users to monitor their vehicles in real-time, improving overall security.
- **Facilitate Vehicle Movements:** Integrate motor controls for movement, allowing users to remotely adjust vehicle positioning.
- **Future Expansion Capabilities:** Design the system to support future upgrades, such as AI recognition, GPS tracking, and enhanced automation, for continued relevance and adaptability.

Ultimately, the goal is to create a comprehensive, intelligent, and secure system that elevates the user experience, enhances vehicle safety, and offers greater convenience in managing and monitoring vehicles remotely.

1.4 Goal of Project

The goal of the Smart WiFi Surveillance System project is to create a comprehensive and innovative solution that transforms vehicle security, monitoring, and control by leveraging advanced Internet of Things (IoT) technologies. The primary objective is to develop a system that allows vehicle owners to monitor, control, and interact with their vehicles remotely and in real-time using reliable wireless communication. By integrating the ESP32 WiFi module, ESP32 Camera Module, L298 motor driver, and four motors, this system aims to offer key functionalities such as continuous live video surveillance and real-time vehicle movement control.

This system will enable users to access live video feeds from the vehicle’s camera, allowing for continuous monitoring of the vehicle's surroundings. This enhances security and provides peace of mind, whether the vehicle is parked or in motion. Users can maintain situational awareness at all times, which helps deter potential threats and ensures immediate action can be taken if needed.

Additionally, the system allows users to remotely control the vehicle’s movements—including steering, speed, and positioning—through an intuitive web interface or mobile app. This capability provides flexibility in vehicle operation, ensuring that users have comprehensive control over their vehicle even when they are not physically present.

The Smart WiFi Surveillance System is designed to offer a seamless and user-friendly experience, making the platform easy to navigate and efficient to use. This ensures a smooth interaction between the user and the vehicle, regardless of their location. Ultimately, the project aims to deliver a solution that combines advanced vehicle surveillance with automated control, enhancing security and providing a more connected and intelligent driving experience for individual vehicle owners.

CHAPTER - 2

PROBLEM IDENTIFICATION

2.1 Existing System

Current vehicle security and monitoring systems primarily focus on basic features such as alarms, immobilizers, and GPS-based tracking. However, these systems often lack the ability to provide real-time video surveillance, remote control capabilities, or comprehensive integration with smart technologies. Below are some key types of existing systems:

- **Traditional Vehicle Alarm Systems:** These systems use sensors to detect unauthorized access, motion, or tampering and trigger alarms. While effective at alerting vehicle owners of potential theft or break-ins, they do not offer real-time remote monitoring or control, limiting their functionality. They also lack integration with modern communication technologies, such as mobile apps or IoT devices, which reduces their effectiveness in today's connected world.

- **GPS Tracking Systems:** GPS tracking is one of the most common forms of vehicle security. These systems provide real-time location data to users, enabling them to track their vehicle's movements in case of theft. While effective for locating stolen vehicles, GPS trackers do not offer surveillance, video streaming, or remote vehicle control, meaning they do not fully address the growing need for comprehensive monitoring or real-time interaction with the vehicle.

- **Remote Start/Control Systems:** Some advanced vehicles offer remote start/stop or movement control features through mobile apps or key fobs. These systems allow users to remotely start their car or control certain functions. However, they typically do not provide surveillance features like live video feeds, and they often rely on short-range communication (such as Bluetooth) or limited connectivity, which restricts their usefulness over longer distances.

- **Dash Cameras:** Dash cameras are widely used for recording video footage while driving or when the vehicle is parked. Many modern dash cams offer features such as motion detection or parking mode, which can capture footage when the vehicle is stationary. However, most dash cameras lack live-streaming capabilities, remote access, or integration with other vehicle systems, such as movement control. Additionally, while they offer video surveillance, they do not provide real-time, interactive vehicle management or secure remote control.

2.2 Proposed System

The Smart WiFi Surveillance System is an advanced solution aimed at enhancing vehicle security, monitoring, and control by utilizing modern IoT technologies. The system integrates key components, including the ESP32 WiFi module, ESP32 Camera Module, and L298 motor driver, to provide real-time surveillance and remote control of vehicle movement. These features combine to offer a comprehensive platform for vehicle owners to maintain oversight and control with ease and reliability.

The ESP32 WiFi Module serves as the central hub, enabling seamless wireless communication between the vehicle and external networks. This module allows users to remotely access and manage their vehicle through a web interface or mobile app, ensuring real-time data transmission and connectivity for continuous monitoring and remote control. The module's connectivity facilitates enhanced vehicle interaction from any location.

The ESP32 Camera Module captures live video feeds of the vehicle’s surroundings, supporting continuous surveillance and security. Users can remotely access the video stream, ensuring that the vehicle remains under watch even when not actively monitored. The camera’s motion detection feature further enhances security by triggering alerts for unusual activity, enabling prompt responses to potential threats.

The system's L298 Motor Driver, connected to four DC motors, allows for precise remote control of the vehicle's movement. Users can steer the vehicle forward, backward, left, or right through a user-friendly interface. This capability adds flexibility for tasks such as repositioning the vehicle or navigating confined spaces, boosting both convenience and security.

Additionally, the system includes an intuitive web interface that provides easy access to live video feeds, movement controls, and system settings. Real-time notifications keep users informed of any irregular activity, ensuring that they can take immediate action when necessary. In summary, the Smart WiFi Surveillance System integrates advanced surveillance and control features, offering vehicle owners a reliable solution for enhanced security and seamless remote interaction.

CHAPTER 3

REQUIREMENTS

3.1 Software Requirements

- **Arduino IDE:** The Integrated Development Environment (IDE) used for programming the ESP32-CAM module. It allows for coding, compiling, and uploading the program to the module.

- **Embedded C Language:** The programming language used to implement the logic for controlling the ESP32-CAM, motor driver, and other components of the system.

- **i5 Processor Computer:** A computer with an Intel i5 processor or equivalent is recommended to ensure smooth operation of the Arduino IDE and efficient coding and uploading of the program.

- **CH340 Drivers:** These drivers enable serial communication between the ESP32-CAM module and the computer, allowing for code uploading and communication during development.

3.2 Hardware Requirements

- **ESP32-CAM Module and WiFi Module:**
  The ESP32-CAM integrates both the ESP32 WiFi module and an OV2640 camera. It captures live video and transmits it over a WiFi network, enabling real-time surveillance and remote monitoring.

- **L298N Motor Driver:**

  A dual H-Bridge motor driver used to control the four DC motors. It manages the direction and speed of the motors, allowing movement of the vehicle in various directions (forward, backward, left, right).

- **Four DC Motors:**

  Mounted on a chassis, these motors provide mobility for the vehicle. They are controlled by the L298N motor driver to allow the vehicle to move autonomously or under remote control.

- **Battery Pack:**

  The battery pack powers the entire system, including the ESP32-CAM module, motor driver, and motors, ensuring the vehicle operates independently without external power.

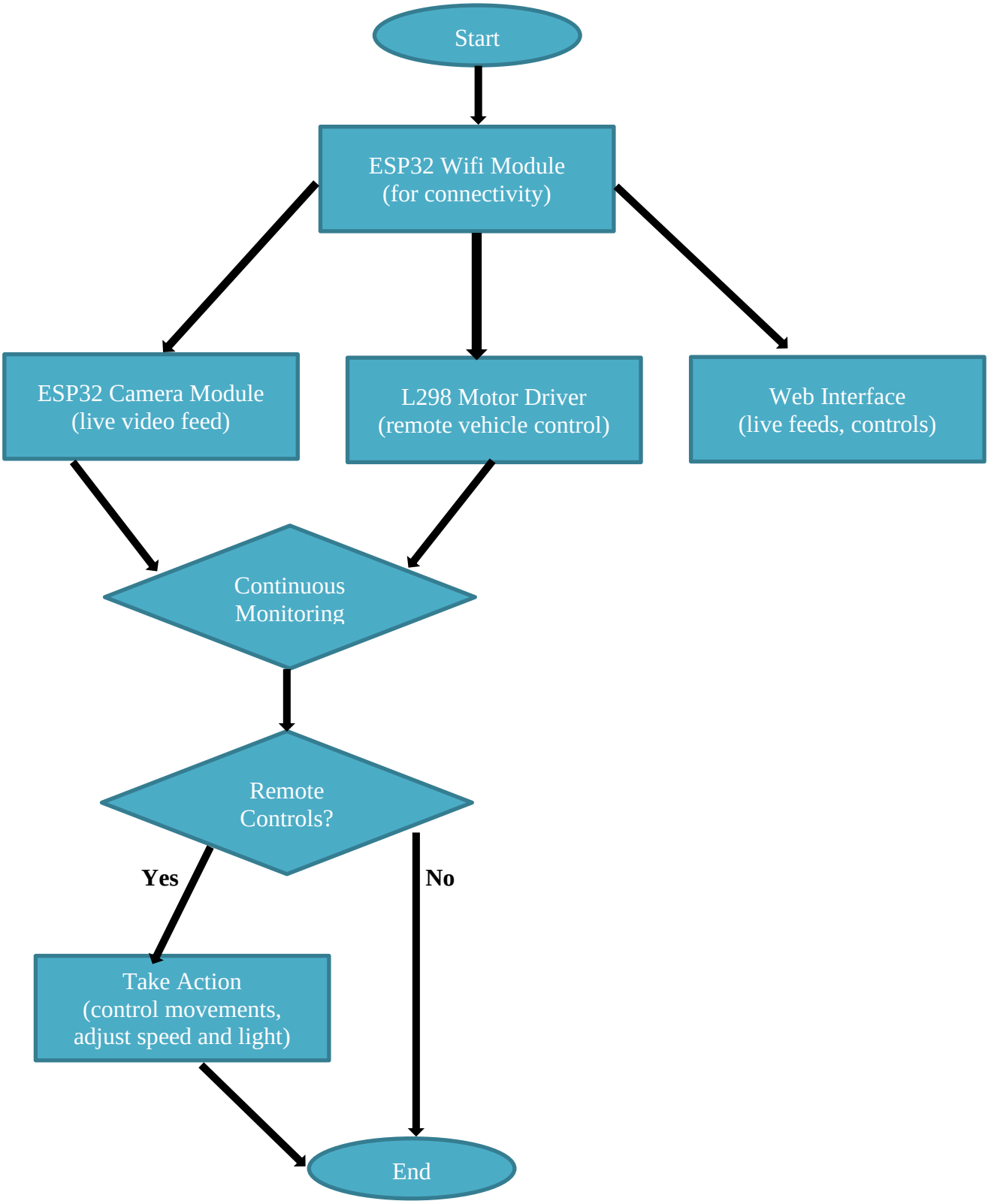
- **Connecting Wires:**

  These wires are used to connect all hardware components, ensuring proper electrical connections for signal transmission and power distribution.

CHAPTER 4

DESIGN AND IMPLEMENTATION

4.1 Design



Design Overview

1.Start

This is the beginning point of the system's operation, initializing the ESP32 WiFi module and other components.

2. ESP32 WiFi Module (for Connectivity)

The ESP32 module serves as the central hub, providing WiFi connectivity for the system. It manages communication between the camera, motor driver, and web interface, enabling remote monitoring and control.

3. Core Components

From the ESP32 module, the flowchart branches out to three main components:

- **ESP32 Camera Module (Live Video Feed):**
  Captures live video feeds and detects motion around the vehicle. This enables real-time surveillance, sending data to the web interface.
- **L298 Motor Driver (Remote Vehicle Control):**
  Controls the vehicle’s movement, allowing users to adjust speed, direction, and even lighting as needed. This provides flexibility for remote positioning and lighting adjustments.
- **Web Interface (Live Feeds, Controls):**
  Acts as the user interface, accessible via a web server. It displays live video feeds, provides controls for vehicle movement, and sends notifications when motion is detected or specific actions are triggered.

4. Continuous Monitoring

The system continuously monitors surroundings for any motion or changes and allows for real-time adjustments to the vehicle’s position, speed, and lighting.

5. Remote Controls?

A decision point where the system checks if the user has taken any remote action in response to alerts or the live feed.

If Yes, the flow proceeds to the action phase; if No, it continues monitoring.

6. Take Action (Control Movements, Adjust Speed and Light)

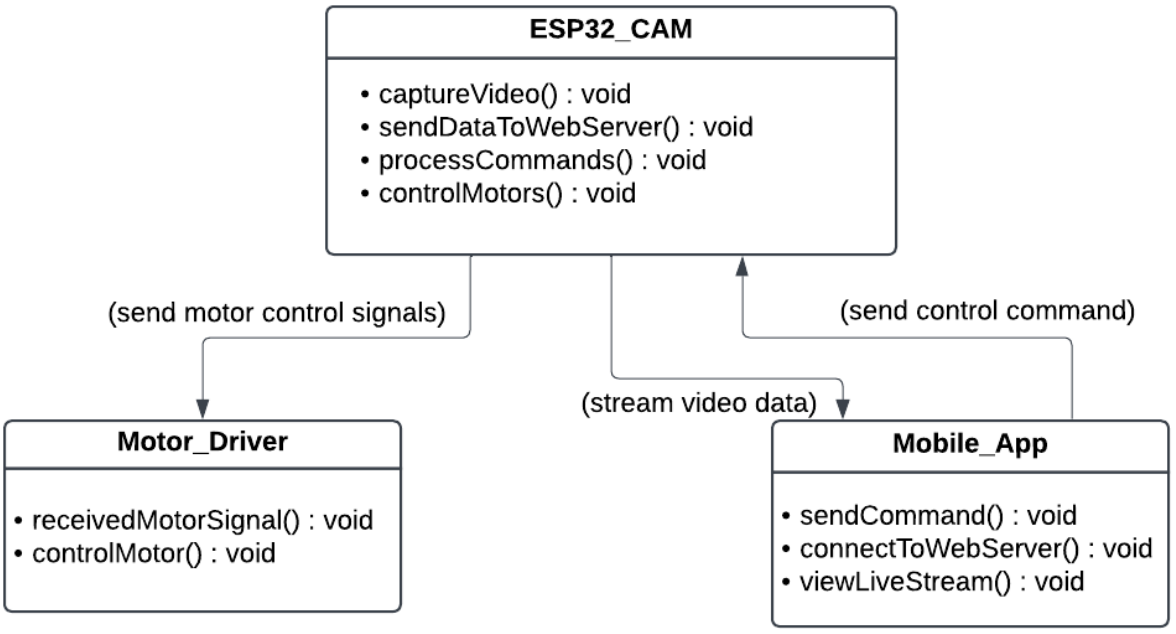
Based on user input, the system executes actions such as adjusting the vehicle's position, modifying speed, or altering light intensity. This level of control enhances the system's responsiveness to real-time situations, optimizing security and adaptability.

7. End

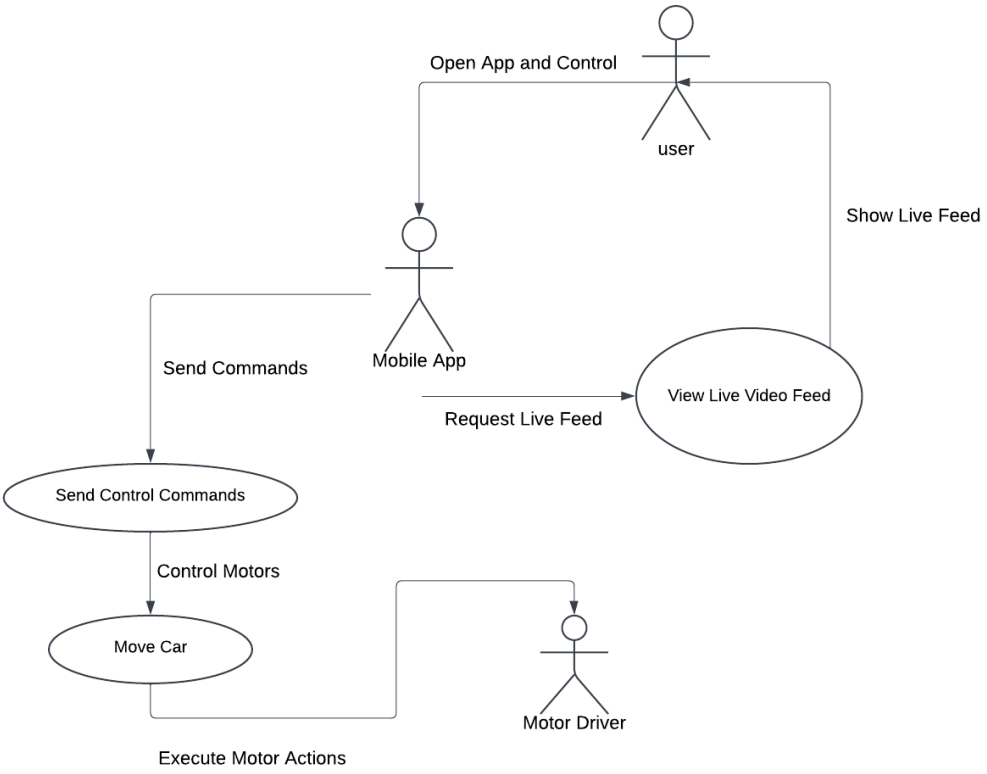
This signifies the completion of the current monitoring or control cycle, looping back to continuous monitoring as long as the system is active.

UML Diagrams

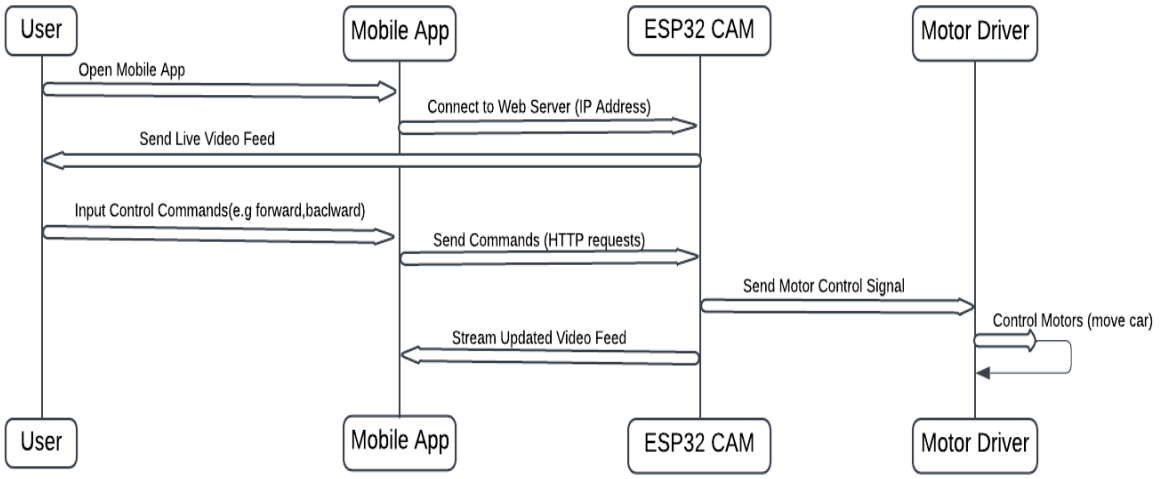
- **Class Diagram**



- **Use Case Diagram**



- **Sequence Diagram**



4.2 Implementation

1. Hardware Setup

- **ESP32-CAM Module Setup**

  Positioning: The ESP32-CAM module is positioned at the front of the vehicle, facing forward. This allows the system to capture real-time video feeds.

  Camera Initialization: The camera module connects to the ESP32 through GPIO pins. In the software, you'll need to configure the camera for streaming the video feed. The ESP32-CAM will be responsible for capturing the video and sending it to a remote viewer over WiFi.

  Connection to Power: The ESP32-CAM is powered using the 5V output from the Battery Pack.

- **L298N Motor Driver Setup**

  Positioning: The L298N Motor Driver is positioned at the center of the vehicle, connected to the four DC motors. The motor driver is essential for controlling the vehicle’s movement by managing the power supplied to each motor.

  Control: The L298N uses PWM (Pulse Width Modulation) signals sent from the ESP32 to control the speed and direction of the motors. The four motors are responsible for the vehicle's movement: two motors control the left and right wheels, and two control forward/backward movement.

  Wiring: The motor driver is connected to the ESP32’s GPIO pins, which are configured to control each motor's direction (forward, backward, left, right). The L298N module is also connected to the Battery Pack, providing sufficient power to drive the motors.

- **Battery Pack Setup**

  Power Supply: Two rechargeable batteries provide portable power to the ESP32-CAM, the motor driver, and the DC motors. These batteries are typically lithium-ion batteries, which offer a good balance of weight and capacity for mobile applications.

  Voltage Regulation: The batteries provide power at a voltage level that is compatible with both the ESP32 and the L298N motor driver, typically around 7.4V or 12V.

- **Wiring**

  Connections: The wiring connects the ESP32-CAM, the L298N motor driver, and the battery pack. The connections are made through jumper wires, ensuring that the voltage and signals are directed properly from one component to another.

  Minimizing Clutter: The wiring should be organized and well-secured to prevent interference or damage during operation, ensuring stable connections and reliable functionality.

2. Software Setup

- **Arduino IDE Configuration**

  Installation: Download and install the Arduino IDE on a computer with at least an i5 Processor to ensure smooth performance while programming and uploading the code to the ESP32.

  ESP32 Board Setup: To program the ESP32-CAM, you must add the ESP32 board package to the Arduino IDE. Navigate to File > Preferences, then add the ESP32 board URL to the “Additional Boards Manager URLs” section. Install the board from the Boards Manager.

- **Embedded C Programming**

  ESP32-CAM Code: The program is written in Embedded C, which allows the ESP32-CAM to connect to a WiFi network, capture video, and control the vehicle's movements. The main tasks of the code include:

  Connecting the ESP32-CAM to a WiFi network using WiFi.begin().

  Capturing video from the camera using the ESP32CAM library.

  Sending the live video feed to a remote web browser.

  Reading the vehicle control commands from the web interface and controlling the motors accordingly.

- **Web Interface**

  HTML and JavaScript: To control the vehicle remotely, a simple web interface is created using HTML and JavaScript. This interface allows the user to view the live video feed from the ESP32-CAM and send commands to move the vehicle forward, backward, left, or right.

3. Testing and Debugging

Step 1: System Testing

WiFi Connection: Ensure that the ESP32 successfully connects to the WiFi network and that the video feed is accessible through the web interface.

Motor Functionality: Test the forward, backward, left, and right commands to confirm that the motors are responding correctly to the signals.

Camera Stream: Verify that the camera is streaming live video without significant lag. Adjust the camera quality settings if needed to optimize the streaming experience.

Step 2: Debugging

Video Feed Issues: If the camera feed does not work or is choppy, check the connection to the ESP32-CAM and ensure that the camera configuration is set up correctly in the code.

Motor Control Issues: If the motors are not functioning as expected, check the wiring connections and ensure that the correct GPIO pins are used for motor control.

4. Final Integration

Step 1: Assembling the Vehicle

Mount the ESP32-CAM, L298N Motor Driver, and DC motors securely onto the chassis.

Connect the components using appropriate wiring, ensuring all connections are stable and free from potential short circuits.

Test the vehicle to ensure it functions properly, with the camera streaming video and the motors responding to control commands.

Step 2: Deployment

Once everything is working, the vehicle is ready for real-world operation. The system can be used for vehicle surveillance, remote monitoring, or as a prototype for autonomous vehicles.

CHAPTER 5

CODE

5.1 Source Code

```
#include "esp_camera.h"
#include <Arduino.h>
#include <WiFi.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <iostream>
#include <sstream>



#define BLYNK_TEMPLATE_ID "TMPL3ivHc5yjR"
#define BLYNK_TEMPLATE_NAME "CAr"
#define BLYNK_AUTH_TOKEN "NA53IL1ViMT1BuV8I2IanDtCoLRORw_T"
#include <BlynkSimpleEsp32.h> // Include Blynk library
struct MOTOR_PINS
{
  int pinEn;
  int pinIN1;
  int pinIN2;
};

std::vector<MOTOR_PINS> motorPins =
{
  {12, 13, 15},  //RIGHT_MOTOR Pins (EnA, IN1, IN2)
  {12, 14, 2},   //LEFT_MOTOR  Pins (EnB, IN3, IN4)
};
#define LIGHT_PIN 4

#define UP 1
#define DOWN 2
#define LEFT 3
#define RIGHT 4
#define STOP 0

#define RIGHT_MOTOR 0
#define LEFT_MOTOR 1


#define FORWARD 1
```

```
#define BACKWARD -1

const int PWMFreq = 1000; /* 1 KHz */
const int PWMResolution = 8;
const int PWMSpeedChannel = 2;
const int PWMLightChannel = 3;

//Camera related constants
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22
const char* ssid     = "MyWiFiCar";
const char* password = "12345678";
const char* auth = "NA53IL1ViMT1BuV8I2IanDtCoLRORw_T";

AsyncWebServer server(80);
AsyncWebSocket wsCamera("/Camera");
AsyncWebSocket wsCarInput("/CarInput");
uint32_t cameraClientId = 0;

const char* htmlHomePage PROGMEM = R"HTMLHOMEPAGE(
<!DOCTYPE html>
<html>
  <head>
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-
scale=1, user-scalable=no">
    <style>
    .arrows {
```

```
    font-size:40px;
    color:red;
  }


td.button {
    background-color:black;
    border-radius:25%;
    box-shadow: 5px 5px #888888;
  }
  td.button:active {
    transform: translate(5px,5px);
    box-shadow: none;
  }

  .noselect {
    -webkit-touch-callout: none; /* iOS Safari */
      -webkit-user-select: none; /* Safari */
       -khtml-user-select: none; /* Konqueror HTML */
         -moz-user-select: none; /* Firefox */
          -ms-user-select: none; /* Internet Explorer/Edge */
              user-select: none; /* Non-prefixed version, currently
                                    supported by Chrome and Opera */
  }

  .slidecontainer {
    width: 100%;
  }

  .slider {
    -webkit-appearance: none;
    width: 100%;
    height: 15px;
    border-radius: 5px;
    background: #d3d3d3;
    outline: none;
    opacity: 0.7;
    -webkit-transition: .2s;
    transition: opacity .2s;
  }

  .slider:hover {
    opacity: 1;
```

```
    .slider::-webkit-slider-thumb {
      -webkit-appearance: none;
      appearance: none;

      width: 25px;
      height: 25px;
      border-radius: 50%;
      background: red;
      cursor: pointer;
    }

    .slider::-moz-range-thumb {
      width: 25px;
      height: 25px;
      border-radius: 50%;
      background: red;
      cursor: pointer;
    }

    </style>

  </head>
  <body class="noselect" align="center" style="background-color:white">

    <!--h2 style="color: teal;text-align:center;">Wi-Fi Camera &#128663;
Control</h2-->

    <table id="mainTable" style="width:400px;margin:auto;table-layout:fixed"
CELLSPACING=10>
      <tr>
        <img id="cameraImage" src="" style="width:400px;height:250px"></td>
      </tr>
      <tr>
        <td></td>
        <td class="button" ontouchstart='sendButtonInput("MoveCar","1")'
ontouchend='sendButtonInput("MoveCar","0")'><span class="arrows"
>&#8679;</span></td>
        <td></td>
      </tr>
      <tr>




        <td class="button" ontouchstart='sendButtonInput("MoveCar","3")'
```

```
ontouchend='sendButtonInput("MoveCar","0")'><span class="arrows"
>&#8678;</span></td>
        <td class="button"></td>
        <td class="button" ontouchstart='sendButtonInput("MoveCar","4")'
ontouchend='sendButtonInput("MoveCar","0")'><span class="arrows"
>&#8680;</span></td>
      </tr>
      <tr>

        <td></td>
        <td class="button" ontouchstart='sendButtonInput("MoveCar","2")'
ontouchend='sendButtonInput("MoveCar","0")'><span class="arrows"
>&#8681;</span></td>
        <td></td>
      </tr>
      <tr/><tr/>
      <tr>
        <td style="text-align:left"><b>Speed:</b></td>
        <td colspan=2>
         <div class="slidecontainer">
            <input type="range" min="0" max="255" value="150" class="slider"
id="Speed" oninput='sendButtonInput("Speed",value)'>
          </div>
        </td>
      </tr>
      <tr>
        <td style="text-align:left"><b>Light:</b></td>
        <td colspan=2>
          <div class="slidecontainer">
            <input type="range" min="0" max="255" value="0" class="slider"
id="Light" oninput='sendButtonInput("Light",value)'>
          </div>
        </td>
      </tr>
    </table>

    <script>
      var webSocketCameraUrl = "ws:\/\/" + window.location.hostname + "/Camera";
      var webSocketCarInputUrl = "ws:\/\/" + window


.location.hostname + "/CarInput";
      var websocketCamera;
      var websocketCarInput;
```

```
    function initCameraWebSocket()
    {
      websocketCamera = new WebSocket(webSocketCameraUrl);
      websocketCamera.binaryType = 'blob';
      websocketCamera.onopen    = function(event){};
      websocketCamera.onclose   =
function(event){setTimeout(initCameraWebSocket, 2000);};
      websocketCamera.onmessage = function(event)
      {


        var imageId = document.getElementById("cameraImage");
        imageId.src = URL.createObjectURL(event.data);
      };
    }

    function initCarInputWebSocket()
    {
      websocketCarInput = new WebSocket(webSocketCarInputUrl);
      websocketCarInput.onopen    = function(event)
      {
        var speedButton = document.getElementById("Speed");
        sendButtonInput("Speed", speedButton.value);
        var lightButton = document.getElementById("Light");
        sendButtonInput("Light", lightButton.value);
      };
      websocketCarInput.onclose   =
function(event){setTimeout(initCarInputWebSocket, 2000);};
      websocketCarInput.onmessage = function(event){};
    }

    function initWebSocket()
    {
      initCameraWebSocket ();
      initCarInputWebSocket();
    }

    function sendButtonInput(key, value)




{
      var data = key + "," + value;
      websocketCarInput.send(data);
```

```
      }

      window.onload = initWebSocket;
      document.getElementById("mainTable").addEventListener("touchend",
function(event){
        event.preventDefault()
      });
    </script>
  </body>
</html>
)HTMLHOMEPAGE";

// Blynk virtual pin assignments
#define VIRTUAL_PIN_SPEED V0
#define VIRTUAL_PIN_LIGHT V1

void rotateMotor(int motorNumber, int motorDirection)
{
  if (motorDirection == FORWARD)
  {
    digitalWrite(motorPins[motorNumber].pinIN1, HIGH);
    digitalWrite(motorPins[motorNumber].pinIN2, LOW);
  }
  else if (motorDirection == BACKWARD)
  {
    digitalWrite(motorPins[motorNumber].pinIN1, LOW);
    digitalWrite(motorPins[motorNumber].pinIN2, HIGH);
  }
  else
  {
    digitalWrite(motorPins[motorNumber].pinIN1, LOW);
    digitalWrite(motorPins[motorNumber].pinIN2, LOW);
  }
}

void moveCar(int inputValue)
{
  Serial.printf("Got value as %d\n", inputValue);
  switch(inputValue)
  {



    case UP:
      rotateMotor(RIGHT_MOTOR, FORWARD);
```

```
      rotateMotor(LEFT_MOTOR, FORWARD);
      break;

    case DOWN:
      rotateMotor(RIGHT_MOTOR, BACKWARD);
      rotateMotor(LEFT_MOTOR, BACKWARD);
      break;

    case LEFT:
      rotateMotor(RIGHT_MOTOR, FORWARD);
      rotateMotor(LEFT_MOTOR, BACKWARD);
      break;

    case RIGHT:
      rotateMotor(RIGHT_MOTOR, BACKWARD);
      rotateMotor(LEFT_MOTOR, FORWARD);
      break;


    case STOP:
      rotateMotor(RIGHT_MOTOR, STOP);
      rotateMotor(LEFT_MOTOR, STOP);
      break;

    default:
      rotateMotor(RIGHT_MOTOR, STOP);
      rotateMotor(LEFT_MOTOR, STOP);
      break;
  }
}

void handleRoot(AsyncWebServerRequest *request)
{
  request->send_P(200, "text/html", htmlHomePage);
}

void handleNotFound(AsyncWebServerRequest *request)
{
    request->send(404, "text/plain", "File Not Found");
}




void onCarInputWebSocketEvent(AsyncWebSocket *server,
```

```
                    AsyncWebSocketClient *client,
                    AwsEventType type,
                    void *arg,
                    uint8_t *data,
                    size_t len)
{
  switch (type)
  {
    case WS_EVT_CONNECT:
      Serial.printf("WebSocket client #%u connected from %s\n", client->id(), client-
>remoteIP().toString().c_str());
      break;
    case WS_EVT_DISCONNECT:
      Serial.printf("WebSocket client #%u disconnected\n", client->id());
      moveCar(0);
      ledcWrite(PWMLightChannel, 0);
      break;
    case WS_EVT_DATA:
      AwsFrameInfo *info;
      info = (AwsFrameInfo*)arg;
      if (info->final && info->index == 0 && info->len == len && info->opcode ==
WS_TEXT)

      {
        std::string myData = "";
        myData.assign((char *)data, len);
        std::istringstream ss(myData);
        std::string key, value;
        std::getline(ss, key, ',');
        std::getline(ss, value, ',');
        Serial.printf("Key [%s] Value[%s]\n", key.c_str(), value.c_str());
        int valueInt = atoi(value.c_str());
        if (key == "MoveCar")
        {
          moveCar(valueInt);
        }
        else if (key == "Speed")
        {
          ledcWrite(PWMSpeedChannel, valueInt);
          Blynk.virtualWrite(VIRTUAL_PIN_SPEED, valueInt); // Update Blynk
virtual pin with speed value
        }
        else if (key == "Light")
```

```
 {
        ledcWrite(PWMLightChannel, valueInt);
        Blynk.virtualWrite(VIRTUAL_PIN_LIGHT, valueInt); // Update Blynk
virtual pin with light value
       }
     }
     break;
   case WS_EVT_PONG:
   case WS_EVT_ERROR:
     break;
   default:
     break;
  }
}

void onCameraWebSocketEvent(AsyncWebSocket *server,
                    AsyncWebSocketClient *client,
                    AwsEventType type,
                    void *arg,
                    uint8_t *data,
                    size_t len)
{
  switch (type)
  {


   case WS_EVT_CONNECT:
     Serial.printf("WebSocket client #%u connected from %s\n", client->id(), client-
>remoteIP().toString().c_str());
     cameraClientId = client->id();
     break;
   case WS_EVT_DISCONNECT:
     Serial.printf("WebSocket client #%u disconnected\n", client->id());
     cameraClientId = 0;
     break;
   case WS_EVT_DATA:
     break;
   case WS_EVT_PONG:
   case WS_EVT_ERROR:
     break;
   default:
     break;
  }
}
```

```
void setupCamera()
{
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;




  config.frame_size = FRAMESIZE_VGA;
  config.jpeg_quality = 10;
  config.fb_count = 1;

  // camera init
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK)
  {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  if (psramFound())
  {
    heap_caps_malloc_extmem_enable(20000);
    Serial.printf("PSRAM initialized. malloc to take memory from psram above this
size");
```

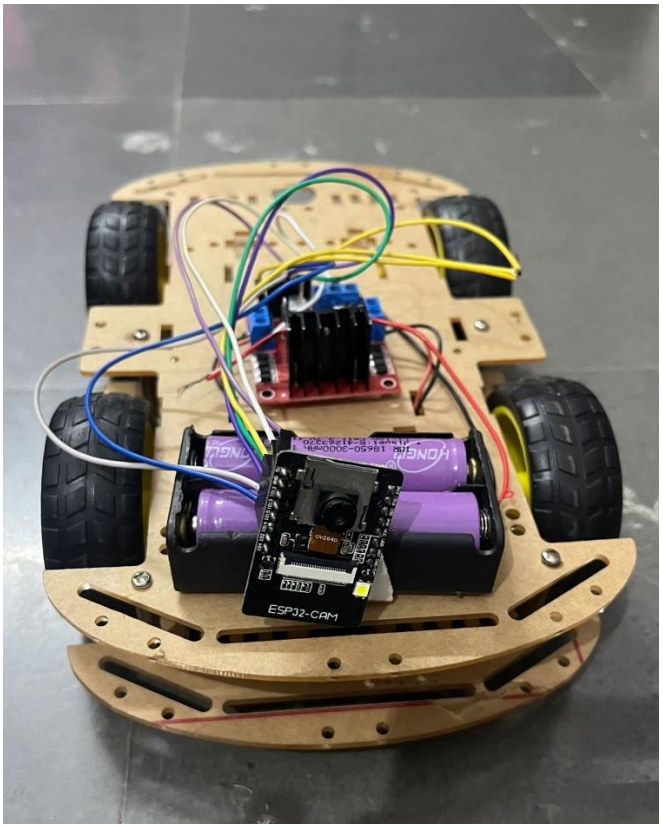
```
  }
}

void sendCameraPicture()
{
  if (cameraClientId == 0)
  {
    return;
  }
  unsigned long  startTime1 = millis();
  //capture a frame
  camera_fb_t * fb = esp_camera_fb_get();
  if (!fb)
  {
      Serial.println("Frame buffer could not be acquired");
      return;
  }

  unsigned long  startTime2 = millis();
  wsCamera.binary(cameraClientId, fb->buf, fb->len);
  esp_camera_fb_return(fb);

  //Wait for message to be delivered
  while (true)
  {
    AsyncWebSocketClient * clientPointer = wsCamera.client(cameraClientId);
    if (!clientPointer || !(clientPointer->queueIsFull()))
    {

      break;
    }
    delay(1);
  }

  unsigned long  startTime3 = millis();
  Serial.printf("Time taken Total: %d|%d|%d\n",startTime3 - startTime1, startTime2
- startTime1, startTime3-startTime2 );
}

void setUpPinModes()
{
```

```
//Set up PWM
  ledcSetup(PWMSpeedChannel, PWMFreq, PWMResolution);
  ledcSetup(PWMLightChannel, PWMFreq, PWMResolution);
  for (int i = 0; i < motorPins.size(); i++)
  {
    pinMode(motorPins[i].pinEn, OUTPUT);
    pinMode(motorPins[i].pinIN1, OUTPUT);
    pinMode(motorPins[i].pinIN2, OUTPUT);

    /* Attach the PWM Channel to the motor enb Pin */
    ledcAttachPin(motorPins[i].pinEn, PWMSpeedChannel);
  }
  moveCar(STOP);

  pinMode(LIGHT_PIN, OUTPUT);
  ledcAttachPin(LIGHT_PIN, PWMLightChannel);
}

void setup(void)
{
  setUpPinModes();
  Serial.begin(115200);

  WiFi.softAP(ssid, password);
  IPAddress IP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(IP);

  Blynk.begin(auth, ssid, password); // Initialize Blynk with authentication token
and Wi-Fi credentials



  server.on("/", HTTP_GET, handleRoot);
  server.onNotFound(handleNotFound);

  wsCamera.onEvent(onCameraWebSocketEvent);
  server.addHandler(&wsCamera);

  wsCarInput.onEvent(onCarInputWebSocketEvent);
  server.addHandler(&wsCarInput);

  server.begin();
  Serial.println("HTTP server started");
```

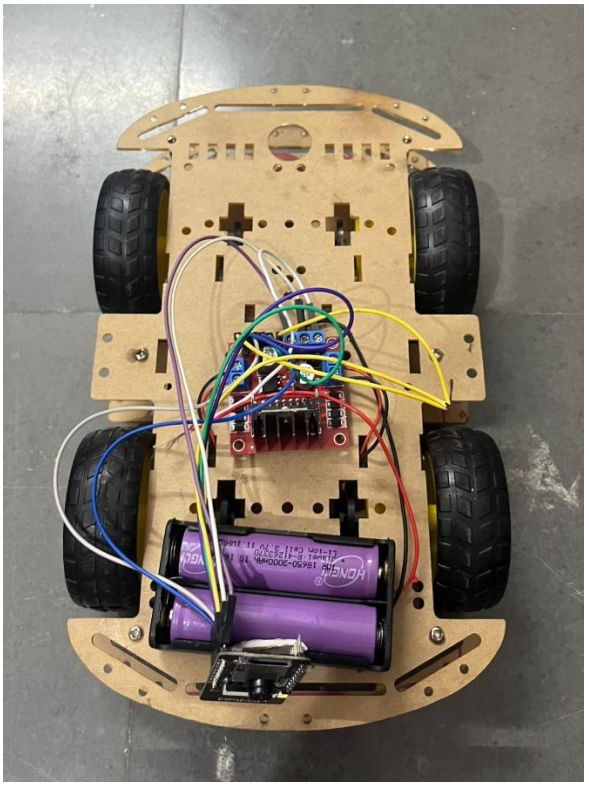
```
  setupCamera();
}

void loop()
{
  Blynk.run(); // Run Blynk tasks
  wsCamera.cleanupClients();
  wsCarInput.cleanupClients();
  sendCameraPicture();
  Serial.printf("SPIRam Total heap %d, SPIRam Free Heap %d\n",
ESP.getPsramSize(), ESP.getFreePsram());
}
```

5.2 Screenshot of Application

Hardware Setup of the Surveillance Vehicle



Web Interface for Control and Video Streaming



CHAPTER 6

RESULTS & CONCLUSION

6.1 Result

The Smart WiFi Surveillance System demonstrated highly effective performance across its various components, achieving the primary goals set for the project. The integration of the ESP32-CAM module provided stable, high-quality real-time video streaming, ensuring continuous surveillance and giving users a clear and reliable view of the vehicle's surroundings. The video feed was accessible through a user-friendly web interface, which functioned smoothly across multiple devices, such as computers and smartphones, enhancing user convenience.

The system's vehicle movement control, powered by the L298N motor driver and four DC motors, delivered smooth and precise directional capabilities. The system responded promptly to user commands for movement (forward, backward, left, and right) without noticeable delays, allowing for easy remote navigation. The motors operated consistently, enabling the vehicle to traverse different surfaces without performance issues, which validated the system's capability for real-world application.

The WiFi connection was another strong aspect, maintaining a stable and reliable link between the vehicle and the control interface. This ensured uninterrupted communication for both live video streaming and movement control, which is essential for real-time security and surveillance tasks. Additionally, the power supply performed reliably, with the battery pack providing enough energy to operate both the ESP32-CAM module and the motors for hours without frequent recharging. This feature supports continuous operation, even in remote or less accessible areas.

The intuitive web interface further strengthened the system's usability, allowing for easy access to live video feeds and movement control within a single, coherent platform. This streamlined interaction made the system efficient and user-friendly. Overall, the project succeeded in its goal of delivering a robust and functional solution for vehicle security and remote monitoring. The combination of video surveillance, movement control, and reliable connectivity showcased the effectiveness of integrating IoT technologies, ultimately enhancing vehicle security and user interaction.

6.2 Conclusion

In conclusion, the Smart WiFi Surveillance System successfully met its primary objectives of real-time vehicle monitoring, enhanced security, and remote control. Through the seamless integration of advanced IoT technologies, the system demonstrated reliability and efficiency. The ESP32-CAM module provided continuous live video streaming, ensuring real-time surveillance and offering users a clear and accurate view of their vehicle's surroundings. The L298N motor driver and four DC motors allowed for smooth and precise control of vehicle movement, enabling users to navigate the vehicle remotely with ease. The stable WiFi connectivity ensured uninterrupted communication between the vehicle and the control interface, vital for the real-time operation of both video streaming and movement control.

The user-friendly web interface further enhanced the system's effectiveness, allowing for intuitive control of the vehicle while simultaneously viewing live video feeds. This seamless interaction between the surveillance system and vehicle control makes it easy for users to monitor and manage their vehicles remotely, increasing both security and convenience. The system performed reliably with stable connectivity, efficient video streaming, and smooth vehicle operation, demonstrating its potential as an effective tool for real-time vehicle surveillance and remote management.

Looking ahead, the successful implementation of this system lays a solid foundation for future innovations in smart vehicle technology. Incorporating features like AI-based facial recognition, advanced GPS tracking, and 5G connectivity could further elevate the system's capabilities, making vehicles more intelligent, secure, and autonomous. These enhancements would expand the system's potential applications, offering smarter, more responsive solutions for vehicle security and management. The Smart WiFi Surveillance System represents a significant step toward the future of connected transportation, providing a glimpse into the next generation of intelligent vehicle technology.

Ultimately, this project serves as a crucial milestone in the rapidly evolving field of smart transportation. By combining IoT, real-time surveillance, and remote vehicle control, it has created a system that not only enhances vehicle security but also offers greater flexibility, user control, and peace of mind. With continuous advancements, the future of connected vehicles looks promising, and the Smart WiFi Surveillance System is at the forefront of these innovations.

REFERENCES

1. Verma, K., Charan, G. S., Pande, A., Abdalla, Y. A., Marshiana, D., & Choubey, C. K. (2023). Internet regulated ESP32 cam robot. 2023 7th International Conference on Computing, Communication, Control and Automation (ICCUBEA), Pune, India, 1-5. https://doi.org/10.1109/ICCUBEA58933.2023.10391981

2. K, S. G., Patel, R. K., Maitra, S., Bhattacharya, S., Moosa, S., & Pavan, P. (2023). Robotic car using NodeMCU ESP8266 Wi-Fi module. 2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 1439-1443. https://doi.org/10.1109/ICACCS57279.2023.10113098

3. Cheng, J., Ye, M., & Zhang, Q. (2022). Design of intelligent remote control car based on Bluetooth. 2022 18th International Conference on Computational Intelligence and Security (CIS), Chengdu, China, 205-209. https://doi.org/10.1109/CIS58238.2022.00050

4. Kaur, A., Jadli, A., Sadhu, A., Goyal, S., Mehra, A., & Rahul. (2021). Cloud-based surveillance using ESP32 CAM. 2021 International Conference on Intelligent Technology, System and Service for Internet of Everything (ITSS-IoE), Sana'a, Yemen, 1-5. https://doi.org/10.1109/ITSS-IoE53029.2021.9615334

5. Constantin, F., Suciu, G., Bosoc, S., & Craciunescu, R. (2020). Internet controlled car. 2020 13th International Conference on Communications (COMM), Bucharest, Romania, 261-264. https://doi.org/10.1109/COMM48946.2020.9141988